

Obstacles for a Llama

Лама хочет путешествовать по одному из регионов Андского плато. У неё есть карта региона в виде сетки из $N \times M$ квадратных клеток. Строки карты пронумерованы сверху вниз от 0 до $N - 1$, а столбцы — слева направо от 0 до $M - 1$. Клетка карты в строке i и столбце j ($0 \leq i < N, 0 \leq j < M$) обозначается как (i, j) .

Лама изучила климат региона и обнаружила, что все клетки в каждой строке карты имеют одинаковую **температуру**, а все клетки в каждом столбце карты имеют одинаковую **влажность**. Лама дала вам два целочисленных массива T и H длиной N и M соответственно. Здесь $T[i]$ ($0 \leq i < N$) содержит температуру клеток в строке i , а $H[j]$ ($0 \leq j < M$) — влажность клеток в столбце j .

Лама также изучила флору плато и заметила, что клетка (i, j) **свободна от растительности** тогда и только тогда, когда ее температура больше её влажности, формально $T[i] > H[j]$.

Лама может перемещаться по плато только по **допустимым путям**. Допустимый путь — это последовательность различных клеток, удовлетворяющих следующим условиям:

- Каждая пара последовательных клеток в пути имеет общую сторону.
- Все клетки на пути свободны от растительности.

Ваша задача — ответить на Q запросов. Для каждого запроса вам даны четыре целых числа: L, R, S и D .

Вы должны определить, существует ли допустимый путь, такой что:

- Путь начинается в клетке $(0, S)$ и заканчивается в клетке $(0, D)$.
- Все клетки в пути лежат в столбцах с L по R , включительно.

Гарантируется, что клетки $(0, S)$ и $(0, D)$ свободны от растительности.

Детали реализации

Первой функцией, которую вы должны реализовать, является:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : массив длины N , содержащий температуру для каждой строки.

- H : массив длины M , содержащий влажность для каждого столбца.
- Эта функция вызывается ровно один раз для каждого теста, перед всеми вызовами `can_reach`.

Вторая функция, которую вы должны реализовать, это:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : целые числа, описывающие запрос.
- Эта функция вызывается Q раз в каждом тесте.

Эта функция должна возвращать `true` тогда и только тогда, когда существует допустимый путь от клетки $(0, S)$ до клетки $(0, D)$, причем все клетки этого пути лежат в столбцах с L по R , включительно.

Ограничения

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ для каждого i , такого что $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ для каждого j , такого что $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S, D \leq R$
- Обе клетки $(0, S)$ и $(0, D)$ свободны от растительности.

Подзадачи

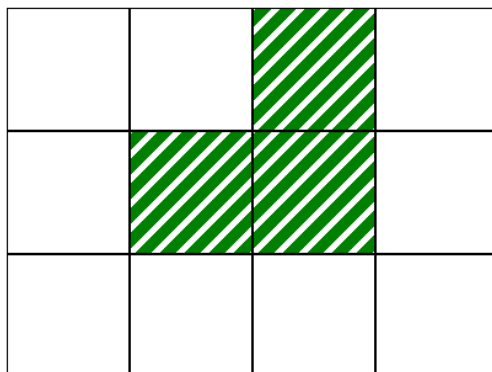
Подзадача	Баллы	Дополнительные ограничения
1	10	$L = 0, R = M - 1$ для каждого запроса. $N = 1$.
2	14	$L = 0, R = M - 1$ для каждого запроса. $T[i - 1] \leq T[i]$ для каждого i , такого что $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ для каждого запроса. $N = 3$ и $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ для каждого запроса. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ для каждого запроса.
6	17	Без дополнительных ограничений.

Пример

Рассмотрим следующий вызов:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

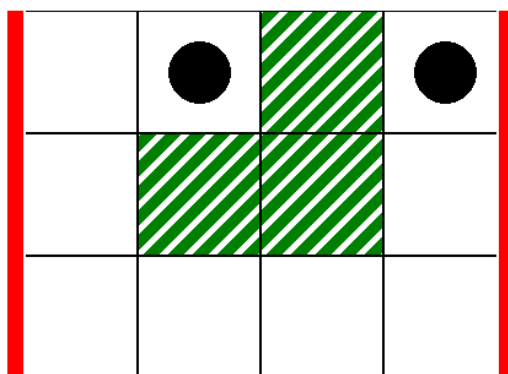
Это соответствует карте на следующем рисунке, где белые клетки свободны от растительности:



В качестве первого запроса рассмотрим следующий вызов:

```
can_reach(0, 3, 1, 3)
```

Это соответствует ситуации на следующем рисунке, где толстые вертикальные линии указывают на диапазон столбцов от $L = 0$ до $R = 3$, а черные круги — на начальную и конечную клетки:



В этом случае лама может попасть из клетки $(0,1)$ в клетку $(0,3)$ по следующему допустимому пути:

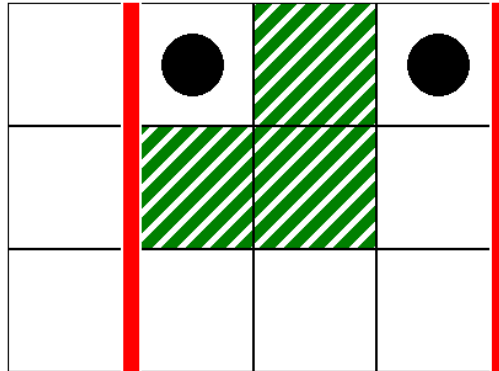
$(0,1), (0,0), (1,0), (2,0), (2,1), (2,2), (2,3), (1,3), (0,3)$

Поэтому этот вызов должен вернуть `true`.

В качестве второго запроса рассмотрим следующий вызов:

```
can_reach(1, 3, 1, 3)
```

Это соответствует сценарию, показанному на следующем рисунке:



В этом случае не существует правильного пути из клетки $(0, 1)$ в клетку $(0, 3)$, такого что все клетки этого пути лежат в столбцах от 1 до 3, включительно. Поэтому этот вызов должен вернуть `false`.

Пример грейдера

Формат ввода:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Здесь $L[k]$, $R[k]$, $S[k]$ и $D[k]$ ($0 \leq k < Q$) задают параметры для каждого вызова `can_reach`.

Формат вывода:

```
A[0]
A[1]
...
A[Q-1]
```

Здесь $A[k]$ ($0 \leq k < Q$) — это 1, если вызов `can_reach(L[k], R[k], S[k], D[k])` возвращает `true`, и 0 — в противном случае.